

Stored Procedures

The JPA 2.1 specification introduced support for calling stored procedures by using the JPA criteria query API. We introduced the `@Procedure` annotation for declaring stored procedure metadata on a repository method.

The examples to follow use the following stored procedure:

Example 1. The definition of the `plus1inout` procedure in HSQL DB.

```
;/
DROP procedure IF EXISTS plus1inout
;/
CREATE procedure plus1inout (IN arg int, OUT res int)
BEGIN ATOMIC
    set res = arg + 1;
END
;/
```

SQL

Metadata for stored procedures can be configured by using the `NamedStoredProcedureQuery` annotation on an entity type.

Example 2. `StoredProcedure` metadata definitions on an entity.

```
@Entity
@NamedStoredProcedureQuery(name = "User.plus1", procedureName = "plus1inout",
parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, name = "arg", type =
Integer.class),
    @StoredProcedureParameter(mode = ParameterMode.OUT, name = "res", type =
Integer.class) })
public class User {}
```

JAVA

Note that `@NamedStoredProcedureQuery` has two different names for the stored procedure. `name` is the name JPA uses. `procedureName` is the name the stored procedure has in the database.

You can reference stored procedures from a repository method in multiple ways. The stored procedure to be called can either be defined directly by using the `value` or `procedureName` attribute of the `@Procedure` annotation. This refers directly to the stored procedure in the database and ignores any configuration via `@NamedStoredProcedureQuery`.

Alternatively you may specify the `@NamedStoredProcedureQuery.name` attribute as the `@Procedure.name` attribute. If neither `value`, `procedureName` nor `name` is configured, the name of the repository method is used as the `name` attribute.

The following example shows how to reference an explicitly mapped procedure:

Example 3. Referencing explicitly mapped procedure with name "plus1inout" in database.

```
@Procedure("plus1inout")
Integer explicitlyNamedPlus1inout(Integer arg);
```

JAVA

The following example is equivalent to the previous one but uses the `procedureName` alias:

Example 4. Referencing implicitly mapped procedure with name "plus1inout" in database via procedureName alias.

```
@Procedure(procedureName = "plus1inout")
Integer callPlus1InOut(Integer arg);
```

JAVA

The following is again equivalent to the previous two but using the method name instead of an explicit annotation attribute.

Example 5. Referencing implicitly mapped named stored procedure "User.plus1" in EntityManager by using the method name.

```
@Procedure
Integer plus1inout(@Param("arg") Integer arg);
```

JAVA

The following example shows how to reference a stored procedure by referencing the `@NamedStoredProcedureQuery.name` attribute.

Example 6. Referencing explicitly mapped named stored procedure "User.plus1IO" in EntityManager.

```
@Procedure(name = "User.plus1IO")
Integer entityAnnotatedCustomNamedProcedurePlus1IO(@Param("arg") Integer arg);
```

JAVA

If the stored procedure getting called has a single out parameter that parameter may be returned as the return value of the method. If there are multiple out parameters specified in a `@NamedStoredProcedureQuery` annotation those can be returned as a `Map` with the key being the parameter name given in the `@NamedStoredProcedureQuery` annotation.

NOTE

Note that if the stored procedure returns a `ResultSet` then any `OUT` parameters are omitted as Java can only return a single method return value unless the method declares a `Map` return type.

The following example shows how to obtain multiple `OUT` parameters if the stored procedure has multiple `OUT` parameters and is registered as `@NamedStoredProcedureQuery`.

`@NamedStoredProcedureQuery` registration is required to provide parameter metadata.

Example 7. StoredProcedure metadata definitions on an entity.

```
@Entity
@NamedStoredProcedureQuery(name = "User.multiple_out_parameters", procedureName =
"multiple_out_parameters", parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, name = "arg", type =
Integer.class),
    @StoredProcedureParameter(mode = ParameterMode.REF_CURSOR, name = "some_cursor", type
= void.class),
    @StoredProcedureParameter(mode = ParameterMode.OUT, name = "res", type =
Integer.class) })
public class User {}
```

JAVA

Example 8. Returning multiple OUT parameters

```
@Procedure(name = "User.multiple_out_parameters")
Map<String, Object> returnsMultipleOutParameters(@Param("arg") Integer arg);
```

JAVA



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the

United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.